# Machine Coding: Tic-Tac-Toe (N×N, winner check)

- **Problem:** Build a Tic-Tac-Toe game on an N×N board with a configurable "K in a row" win rule, winner/draw detection, undo and reset.
- **Tech stack:** React (Vite, JavaScript) (default)
- **Time limit:** 60 minutes (default)
- **Often asked at:** Microsoft, Uber, Atlassian, Swiggy
- **Tests:** grid state, win-check logic

---

## PART A: Thought process

### General steps for any machine coding task

1. **Understand the requirements:** functional, non-functional, clarify, write down assumptions.
2. **List interactions / use cases:** every action and its expected result. This is your test checklist.
3. **Design the architecture:** components / modules with one responsibility each.
4. **Design state:** the minimum state; derive everything else.
5. **Design data structures and contracts:** data shape, function signatures.
6. **Trace the data flow:** one interaction end to end.
7. **Find the hard parts:** solve them on paper before coding.
8. **Build in layers:** simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases.**
10. **Wrap up:** trade-offs and what you would add with more time.

### Time budget (60 minutes)

| Phase | Time |
|---|---|
| Requirements + clarifying questions | 5 min |
| Use cases, architecture, state design | 8 min |
| Hard part on paper (win check) | 5 min |
| Build the core (board, turns, win, draw) | 22 min |
| Layers: undo, N/K config, keyboard a11y | 12 min |
| Edge cases, polish, walk-through | 8 min |

### Tip: think out loud

Say what you are deciding and why: "I'll store the board as a flat array of $N^2$ cells because indexing is `row * N + col` and copying is one `slice()`." The interviewer grades the reasoning as much as the code. If you are

stuck, say what you are trying; silence looks like being lost.

---

# PART B: Apply it to this problem

## 1. Understand the requirements

### Functional requirements

- Render an **N×N** board (default 3×3).
- Two players, **X starts**, turns alternate.
- Click an empty cell to place the current player's mark.
- Detect a **winner**: K marks in a row horizontally, vertically or diagonally (default K = N).
- Detect a **draw**: board full, no winner.
- Show status: next player / winner / draw.
- **Highlight the winning line.**
- Block moves on filled cells and after the game ends.
- **Reset** the game; change **N** and **K** (resets the game).
- **Undo** the last move.

### Non-functional requirements

| Concern | What it means here |
|---|---|
| Performance | Win check after a move must not scan the whole board: check only from the last move, O(N). Only the changed cell re-renders. |
| Scalability | Works for any N (we cap at 10 for UI) and any K ≤ N, e.g. 10×10 with 5 in a row (Gomoku style). |
| Error handling | Invalid N/K input is clamped, never crashes. Illegal moves are ignored in the reducer, not just in the UI. |
| Accessibility | Cells are real `<button>`s with labels ("Row 2, column 3, X"), status announced via `aria-live`, arrow-key navigation with a single Tab stop. |
| Reusability | Game rules are pure functions + a reducer, independent of React UI. Same logic can drive a CLI or a bot. |
| Extensibility | Adding a bot, online play, or more players touches only the reducer / a new hook, not the components. |

### Clarifying questions

1. Is N fixed at 3 or configurable? If configurable, is the win length always N or can it be smaller (K in a row)?
2. Do we need undo / move history (time travel)?
3. Two humans on one device, or player vs computer?
4. Should we detect an early draw (no one can win anymore) or only a full-board draw?

5. Do we need to persist the game or keep a score across rounds?
6. Any requirement on keyboard / screen reader support?

### Assumptions we build with

- N is configurable from 3 to 10; K is configurable from 3 to N (default K = N).
- Two local players, X always starts.
- Draw = board full with no winner (early draw is a "more time" item).
- Undo pops the last move; no redo / persistence / score.
- Changing N or K starts a new game.
- Keyboard and screen reader support are required.

## 2. Identify interactions / use cases

| Action / input | Expected behavior / output |
|---|---|
| App loads | 3×3 empty board, "Next player: X", Undo disabled |
| Click empty cell | Current player's mark placed, turn switches |
| Click filled cell | Nothing happens |
| Move completes K in a row (row / column / diagonal / anti-diagonal) | "Winner: X", winning cells highlighted, board locked |
| Click any cell after a win | Nothing happens |
| Last cell filled, no line | "It's a draw" |
| Click Undo | Last move removed, turn and winner/draw status go back |
| Click Undo on empty board | Disabled, nothing happens |
| Click Reset | Empty board with the same N and K, X to move |
| Change N | New empty N×N board; K clamped to ≤ N |
| Change K | New game with the new win length (clamped 3..N) |
| Enter invalid N (e.g. 50, empty) | Clamped to 3..10 |
| Tab into board | One cell focused (single Tab stop) |
| Arrow keys on board | Focus moves to neighbour cell, stops at edges |
| Enter / Space on focused cell | Plays the move (native button behavior) |

## 3. Design the architecture

| Module | Responsibility |
|---|---|
| `utils/gameLogic.js` | Pure rules: create board, clamp config, find winning line from a move |
| `hooks/useTicTacToe.js` | Reducer (PLAY / UNDO / RESET / CONFIGURE) + derived values (winner, draw, next player) |
| `App.jsx` | Wires the hook into the UI |
| `components/Controls.jsx` | N / K inputs, Undo, Reset |
| `components/Status.jsx` | Next player / winner / draw message ( `aria-live` ) |
| `components/Board.jsx` | Grid layout, roving-tabindex keyboard navigation |
| `components/Cell.jsx` | One memoized button cell |

```
App
 ├── useTicTacToe()  ──uses──>  gameLogic (pure functions)
 ├── Controls   (size, winLength, canUndo, onConfigure, onUndo, onReset)
 ├── Status     (winner, isDraw, currentPlayer)
 └── Board      (board, size, winLine, isOver, onPlay)
      └── Cell × N²   (value, isWinning, isFocused, onPlay)
```

Start with one component that renders a 3×3 grid and a hardcoded 8-line win check. Split into the hook + pure logic once N becomes variable, because that is when the rules become the hard part and deserve their own testable file.

## 4. Design the state / data model

| State or field | Purpose |
|---|---|
| `size` | N, board is N×N |
| `winLength` | K, marks in a row needed to win |
| `history` | Array of snapshots `{ board, winLine, lastMove }`, first one is the empty board |
| `history[i].board` | Flat array of $N^2$ cells: `"X"` , `"O"` or `null` |
| `history[i].winLine` | Indexes of the winning cells, or `null` |
| `history[i].lastMove` | Index of the move that produced this snapshot |

```
const createGame = ({ size = 3, winLength = 3 } = {}) => {
  const config = normalizeConfig(size, winLength);
  return {
    ...config,
    history: [{ board: createBoard(config.size), winLine: null, lastMove: null }],
```

```
  };
};
```

### Senior note: derive, don't store

- **Current player** is not stored: `moveCount % 2 === 0 ? "X" : "O"`, where `moveCount = history.length - 1`. Storing it means two things that can disagree after an undo.
- **Winner** is `board[winLine[0]]`, not a separate field.
- **Draw** is `!winner && moveCount === N * N`.
- **isOver**, **canUndo** are derived too.
- The `winLine` is stored per snapshot (computed once at move time) so undo restores it for free and render never re-runs the check.
- **Ownership:** game state lives in the `useTicTacToe` reducer (single source of truth). The only local UI state is `focusIndex` in `Board` (which cell has the Tab stop), because nothing else cares about it.

## 5. Design the data structures and contracts

### Why a flat array?

```
// 3×3 board after X:0, O:4, X:8
["X", null, null,
 null, "O", null,
 null, null, "X"]
// cell (row, col) <-> index: index = row * N + col
// row = Math.floor(index / N), col = index % N
```

- Copying for an immutable update is one `slice()` (a 2D array needs a nested copy).
- The index is a **stable id** for each cell: perfect as a React `key` (cells never reorder) and as the action payload.
- A `Set` of `winLine` indexes gives O(1) "is this cell winning?" lookups.

### Contracts

```
// utils/gameLogic.js
createBoard(size: number) => (null)[]                        // length size*size
normalizeConfig(size: any, winLength: any) => { size, winLength }  // clamped ints
getWinLine(board, size, winLength, index) => number[] | null  // cells of the line through `index`

// hooks/useTicTacToe.js reducer actions
{ type: "PLAY", index: 4 }
{ type: "UNDO" }
{ type: "RESET" }
{ type: "CONFIGURE", config: { size: 5, winLength: 4 } }

// useTicTacToe(initialConfig) returns
{
```

```
  size: 3, winLength: 3,
  board: ["X", null, ...], winLine: [0, 1, 2] | null, lastMove: 2,
  winner: "X" | null, isDraw: false, isOver: true,
  currentPlayer: "O", canUndo: true,
  play(index), undo(), reset(), configure({ size?, winLength? })
}
```

There is no backend; for online play the same `PLAY { index }` action would be the message sent to the server.

## 6. Trace the data flow

Player X clicks the top-right cell (index 2) on a board where X already has 0 and 1:

```
Click on <Cell index=2>
   |
   ▼
onPlay(2)  →  dispatch({ type: "PLAY", index: 2 })
   |
   ▼
gameReducer
   ├─ current = last history entry
   ├─ guard: winLine exists or board[2] !== null ? return same state
   ├─ player = history.length odd ? "X" : "O"   → "X"
   ├─ board = current.board.slice(); board[2] = "X"
   ├─ winLine = getWinLine(board, 3, 3, 2)       → [2, 1, 0]
   └─ return { ...state, history: [...history, { board, winLine, lastMove: 2 }] }
   |
   ▼
useTicTacToe derives: winner = "X", isOver = true, currentPlayer = "O"
   |
   ▼
Render
   ├─ Status: "Winner: X"  (aria-live announces it)
   ├─ Board: cells 0,1,2 get .cell--win
   └─ Cells: only cells whose props changed re-render (memo)
```

## 7. Important technical challenges

### Challenge 1: Win check that scales to N×N and K in a row

**Problem:** The classic 3×3 solution hardcodes 8 lines. On a 10×10 board with K = 5 there are hundreds of possible lines, and scanning the whole board after every move is $O(N^2 \cdot K)$. Worse, a naive scan that walks `index + 1` can **wrap** from the end of one row to the start of the next and report a fake win (e.g. 4×4, X on cells 3, 4, 5 looks like 3 in a row in the flat array but is two different rows).

**Solution:** Only the move just played can create a new win. From that cell, walk outward in 4 directions (horizontal, vertical, diagonal, anti-diagonal), both ways, while the next cell has the same mark. Work in **(row, col)** with bounds checks, never with raw `index ± 1`, so no wrapping. Cost: O(N) per move.

```
const DIRECTIONS = [[0, 1], [1, 0], [1, 1], [1, -1]];

export function getWinLine(board, size, winLength, index) {
  const player = board[index];
  if (!player) return null;
  const row = Math.floor(index / size);
  const col = index % size;

  for (const [dr, dc] of DIRECTIONS) {
    const line = [index];
    for (const sign of [1, -1]) {
      let r = row + dr * sign;
      let c = col + dc * sign;
      while (r >= 0 && r < size && c >= 0 && c < size && board[r * size + c] === player) {
        line.push(r * size + c);
        r += dr * sign;
        c += dc * sign;
      }
    }
    if (line.length >= winLength) return line;
  }
  return null;
}
```

Returning the **line** (not just `true`) gives us the highlight and the winner (`board[line[0]]`) from one call.

## Challenge 2: Immutable updates, undo, and no "stale" derived state

**Problem:** If you mutate `board[i] = "X"` React does not see a change. If you store `currentPlayer` and `winner` separately, an undo must roll back three things and it is easy to forget one (classic bug: after undoing a winning move the board still says "Winner: X").

**Solution:** Keep a `history` array of snapshots in a reducer. Every move appends a new copied board; undo is `history.slice(0, -1)`. Current player, winner and draw are **derived** from the last snapshot, so they can never be out of sync.

```
case "PLAY": {
  const current = state.history[state.history.length - 1];
  if (current.winLine || current.board[action.index] !== null) return state;
  const player = state.history.length % 2 === 1 ? "X" : "O";
  const board = current.board.slice();
  board[action.index] = player;
  const winLine = getWinLine(board, state.size, state.winLength, action.index);
```

```
    return { ...state, history: [...state.history, { board, winLine, lastMove: action.index }] };
  }
  case "UNDO":
    if (state.history.length === 1) return state;
    return { ...state, history: state.history.slice(0, -1) };
```

The guard lives in the **reducer**, not just in the UI, so a double click, a stale event or a future bot can never play an illegal move. Returning the same `state` object means React skips the re-render.

### Challenge 3: Re-render cost and keyboard accessibility on a big grid

**Problem:** On a 10×10 board every move re-renders 100 cells, and with plain buttons the user must press Tab up to 100 times to reach the Undo button.

**Solution:**

- `React.memo` on `Cell` + stable callbacks (`useCallback` around `dispatch`) so only the changed cell (and the win-line cells at the end) re-render.
- **Roving tabindex:** exactly one cell has `tabIndex=0`; arrow keys move focus inside the grid. The grid is one Tab stop, like a native grid widget.

```
const handleKeyDown = (event) => {
  const step = KEY_STEPS[event.key];         // ArrowUp: [-1, 0], ...
  if (!step) return;
  event.preventDefault();
  const row = Math.floor(activeIndex / size) + step[0];
  const col = (activeIndex % size) + step[1];
  if (row < 0 || row >= size || col < 0 || col >= size) return;
  const next = row * size + col;
  setFocusIndex(next);
  cellRefs.current[next]?.focus();
};
```

Cells use `aria-disabled` instead of `disabled` so they stay focusable (a screen reader user can still read a finished board).

### Challenge 4: Changing N and K safely

**Problem:** The input fires on every keystroke. Typing `""` or `50` must not create a 0×0 or 2500-cell board, and K must never exceed N (a 3×3 board with K = 5 can never be won).

**Solution:** One `normalizeConfig` function clamps both values, and `CONFIGURE` always builds a brand-new game from it.

```
export function normalizeConfig(size, winLength) {
  const n = clamp(Number(size) || MIN_SIZE, MIN_SIZE, MAX_SIZE);
  const k = clamp(Number(winLength) || n, MIN_SIZE, n);
```

```
  return { size: n, winLength: k };
}
```

## 8. Implement in the correct order

| # | Step | Working result |
|---|---|---|
| 1 | Vite app, `App` renders a hardcoded 3×3 grid of buttons with CSS grid | Board visible |
| 2 | `board` state as flat array, click places "X" | Marks appear |
| 3 | Alternate turns, ignore filled cells, show "Next player" | Playable game |
| 4 | Win check (start with 8 hardcoded lines), show winner, lock board | Full 3×3 game |
| 5 | Draw detection | All end states covered |
| 6 | Reset button | Replayable |
| 7 | Move to `useReducer` + `history`; derive player/winner/draw; add Undo | Undo works, no stale status |
| 8 | Extract `gameLogic.js`; replace hardcoded lines with the 4-direction `getWinLine` | Works for any N, K |
| 9 | Controls for N and K with `normalizeConfig`; CSS `--size` variable | N×N board |
| 10 | Highlight winning line | Visual feedback |
| 11 | `memo` Cell, `aria-*`, roving tabindex | Fast + accessible |
| 12 | Node assert self-check for `getWinLine` and the reducer | Logic verified |

## 9. Cover all states and edge cases

| State / edge case | Handling |
|---|---|
| Initial | Empty board, X to move, Undo disabled |
| In progress | Status shows the next player |
| Win (row, column, diagonal, anti-diagonal) | `winLine` set, status "Winner: X", cells highlighted, moves blocked |
| Draw | Board full and no winner: "It's a draw" |
| Win on the very last cell | Checked as a win first ( `isDraw` requires `!winner` ) |
| Click on filled cell | Reducer returns same state, no re-render |
| Click after game over | Reducer guard returns same state |
| Double click / fast clicks | Each click is a separate reducer action against the latest state, no stale closure |

| State / edge case | Handling |
| --- | --- |
| Undo after a win | Previous snapshot has `winLine: null`, game resumes |
| Undo with no moves | Button disabled + reducer guard |
| Line that would wrap across rows in the flat array | Checked in (row, col) with bounds, never `index ± 1` |
| K < N (e.g. 5×5, 3 in a row) | Walk stops at K or more matching cells |
| More than K in a row (e.g. 4 on a K = 3 board) | Counts as a win, full line highlighted |
| N input empty / out of range / not a number | Clamped to 3..10 |
| K > N | Clamped to N |
| N shrinks while focus is on a high index | Board falls back to cell 0 as Tab stop |
| Keyboard at the board edge | Focus stays put |
| Screen reader | Labelled cells, `role="status"` live message |

## 10. Final structure

```
_59_tic_tac_toe/
├── index.html
├── package.json
├── vite.config.js
├── eslint.config.js
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── App.css
    ├── components/
    │   ├── Board.jsx
    │   ├── Cell.jsx
    │   ├── Controls.jsx
    │   └── Status.jsx
    ├── hooks/
    │   └── useTicTacToe.js
    └── utils/
        └── gameLogic.js
```

## 11. Full working code

`src/utils/gameLogic.js`

```
export const MIN_SIZE = 3;
export const MAX_SIZE = 10;

// [rowStep, colStep]: horizontal, vertical, diagonal, anti-diagonal
const DIRECTIONS = [
  [0, 1],
  [1, 0],
  [1, 1],
  [1, -1],
];

export const createBoard = (size) => Array(size * size).fill(null);

const clamp = (value, min, max) => Math.min(Math.max(value, min), max);

export function normalizeConfig(size, winLength) {
  const n = clamp(Number(size) || MIN_SIZE, MIN_SIZE, MAX_SIZE);
  const k = clamp(Number(winLength) || n, MIN_SIZE, n);
  return { size: n, winLength: k };
}

// Only the last move can create a win, so walk out from it in 4 directions: O(N) instead of O(N²).
export function getWinLine(board, size, winLength, index) {
  const player = board[index];
  if (!player) return null;

  const row = Math.floor(index / size);
  const col = index % size;

  for (const [dr, dc] of DIRECTIONS) {
    const line = [index];
    for (const sign of [1, -1]) {
      let r = row + dr * sign;
      let c = col + dc * sign;
      while (r >= 0 && r < size && c >= 0 && c < size && board[r * size + c] === player) {
        line.push(r * size + c);
        r += dr * sign;
        c += dc * sign;
      }
    }
    if (line.length >= winLength) return line;
  }
```

```
  return null;
}
```

`src/hooks/useTicTacToe.js`

```
import { useCallback, useReducer } from "react";
import { createBoard, getWinLine, normalizeConfig } from "../utils/gameLogic.js";

export const createGame = ({ size = 3, winLength = 3 } = {}) => {
  const config = normalizeConfig(size, winLength);
  return {
    ...config,
    history: [{ board: createBoard(config.size), winLine: null, lastMove: null }],
  };
};

export function gameReducer(state, action) {
  switch (action.type) {
    case "PLAY": {
      const current = state.history[state.history.length - 1];
      // Guard: ignore clicks on filled cells or after the game is over
      if (current.winLine || current.board[action.index] !== null) return state;

      const player = state.history.length % 2 === 1 ? "X" : "O";
      const board = current.board.slice();
      board[action.index] = player;
      const winLine = getWinLine(board, state.size, state.winLength, action.index);
      return {
        ...state,
        history: [...state.history, { board, winLine, lastMove: action.index }],
      };
    }
    case "UNDO":
      if (state.history.length === 1) return state;
      return { ...state, history: state.history.slice(0, -1) };
    case "RESET":
      return createGame(state);
    case "CONFIGURE":
      return createGame({ ...state, ...action.config });
    default:
      return state;
  }
}

export function useTicTacToe(initialConfig) {
  const [state, dispatch] = useReducer(gameReducer, initialConfig, createGame);
  const { size, winLength, history } = state;
```

```
  // Everything below is derived, never stored
  const { board, winLine, lastMove } = history[history.length - 1];
  const moveCount = history.length - 1;
  const winner = winLine ? board[winLine[0]] : null;
  const isDraw = !winner && moveCount === size * size;
  const currentPlayer = moveCount % 2 === 0 ? "X" : "O";

  const play = useCallback((index) => dispatch({ type: "PLAY", index }), []);
  const undo = useCallback(() => dispatch({ type: "UNDO" }), []);
  const reset = useCallback(() => dispatch({ type: "RESET" }), []);
  const configure = useCallback((config) => dispatch({ type: "CONFIGURE", config }), []);

  return {
    size,
    winLength,
    board,
    winLine,
    lastMove,
    winner,
    isDraw,
    isOver: Boolean(winner) || isDraw,
    currentPlayer,
    canUndo: moveCount > 0,
    play,
    undo,
    reset,
    configure,
  };
}
```

`src/components/Cell.jsx`

```
import { memo } from "react";

// memo: a move changes one cell, so the other N²-1 cells skip re-render
const Cell = memo(function Cell({ index, value, size, isWinning, isFocused, disabled, onPlay, setRef
}) {
  const row = Math.floor(index / size) + 1;
  const col = (index % size) + 1;

  return (
    <button
      ref={(el) => setRef(index, el)}
      type="button"
      role="gridcell"
      className={`cell ${value ? `cell--${value}` : ""} ${isWinning ? "cell--win" : ""}`}
```

```
      aria-label={`Row ${row}, column ${col}, ${value ?? "empty"}`}
      aria-disabled={disabled || value !== null}
      tabIndex={isFocused ? 0 : -1}
      data-index={index}
      onClick={() => onPlay(index)}
    >
      {value}
    </button>
  );
});

export default Cell;
```

`src/components/Board.jsx`

```
import { useCallback, useRef, useState } from "react";
import Cell from "./Cell";

const KEY_STEPS = {
  ArrowUp: [-1, 0],
  ArrowDown: [1, 0],
  ArrowLeft: [0, -1],
  ArrowRight: [0, 1],
};

export default function Board({ board, size, winLine, isOver, onPlay }) {
  const cellRefs = useRef([]);
  const [focusIndex, setFocusIndex] = useState(0);
  const winSet = new Set(winLine ?? []);
  // Board can shrink after a size change; keep the roving tab stop inside it
  const activeIndex = focusIndex < board.length ? focusIndex : 0;

  const setRef = useCallback((index, el) => {
    cellRefs.current[index] = el;
  }, []);

  // Roving tabindex: the grid is one Tab stop, arrows move inside it
  const handleKeyDown = (event) => {
    const step = KEY_STEPS[event.key];
    if (!step) return;
    event.preventDefault();
    const row = Math.floor(activeIndex / size) + step[0];
    const col = (activeIndex % size) + step[1];
    if (row < 0 || row >= size || col < 0 || col >= size) return;
    const next = row * size + col;
    setFocusIndex(next);
    cellRefs.current[next]?.focus();
```

```
  };

  return (
    <div
      className="board"
      role="grid"
      aria-label={`${size} by ${size} board`}
      style={{ "--size": size }}
      onKeyDown={handleKeyDown}
      onFocus={(e) => e.target.dataset.index && setFocusIndex(Number(e.target.dataset.index))}
    >
      {board.map((value, index) => (
        <Cell
          key={index}
          index={index}
          value={value}
          size={size}
          isWinning={winSet.has(index)}
          isFocused={index === activeIndex}
          disabled={isOver}
          onPlay={onPlay}
          setRef={setRef}
        />
      ))}
    </div>
  );
}
```

`src/components/Controls.jsx`

```
import { MAX_SIZE, MIN_SIZE } from "../utils/gameLogic";

export default function Controls({ size, winLength, canUndo, onConfigure, onUndo, onReset }) {
  return (
    <div className="controls">
      <label>
        Board size (N)
        <input
          type="number"
          min={MIN_SIZE}
          max={MAX_SIZE}
          value={size}
          onChange={(e) => onConfigure({ size: e.target.value, winLength: Math.min(winLength,
Number(e.target.value)) })}
        />
      </label>
      <label>
```

```
        In a row to win (K)
        <input
          type="number"
          min={MIN_SIZE}
          max={size}
          value={winLength}
          onChange={(e) => onConfigure({ winLength: e.target.value })}
        />
      </label>
      <button type="button" onClick={onUndo} disabled={!canUndo}>
        Undo
      </button>
      <button type="button" onClick={onReset}>
        Reset
      </button>
    </div>
  );
}
```

`src/components/Status.jsx`

```
export default function Status({ winner, isDraw, currentPlayer }) {
  let message = `Next player: ${currentPlayer}`;
  if (winner) message = `Winner: ${winner}`;
  else if (isDraw) message = "It's a draw";

  return (
    <p className="status" role="status" aria-live="polite">
      {message}
    </p>
  );
}
```

`src/App.jsx`

```
import Board from "./components/Board";
import Controls from "./components/Controls";
import Status from "./components/Status";
import { useTicTacToe } from "./hooks/useTicTacToe";
import "./App.css";

export default function App() {
  const game = useTicTacToe({ size: 3, winLength: 3 });

  return (
    <main className="app">
```

```
      <h1>Tic Tac Toe</h1>
      <Controls
        size={game.size}
        winLength={game.winLength}
        canUndo={game.canUndo}
        onConfigure={game.configure}
        onUndo={game.undo}
        onReset={game.reset}
      />
      <Status winner={game.winner} isDraw={game.isDraw} currentPlayer={game.currentPlayer} />
      <Board board={game.board} size={game.size} winLine={game.winLine} isOver={game.isOver} onPlay=
{game.play} />
    </main>
  );
}
```

`src/main.jsx`

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`src/App.css`

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f5f6fa;
  color: #1f2333;
}

.app {
  max-width: 520px;
  margin: 0 auto;
  padding: 24px 16px;
  text-align: center;
```

```
}

.controls {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  justify-content: center;
  align-items: end;
}

.controls label {
  display: flex;
  flex-direction: column;
  gap: 4px;
  font-size: 14px;
}

.controls input {
  width: 90px;
  padding: 6px;
}

.controls button {
  padding: 8px 16px;
  cursor: pointer;
}

.status {
  font-size: 20px;
  font-weight: 600;
}

.board {
  display: grid;
  grid-template-columns: repeat(var(--size), 1fr);
  gap: 4px;
  width: min(100%, 480px);
  margin: 0 auto;
}

.cell {
  aspect-ratio: 1;
  font-size: calc(200px / var(--size));
  font-weight: 700;
  border: none;
  border-radius: 6px;
  background: #fff;
```

```
  cursor: pointer;
}

.cell[aria-disabled="true"] {
  cursor: not-allowed;
}

.cell:focus-visible {
  outline: 3px solid #4f6bed;
}

.cell--X {
  color: #e5484d;
}

.cell--O {
  color: #3e63dd;
}

.cell--win {
  background: #c9f2d2;
}
```

## Self-check ( `node selfcheck.mjs` )

```
import assert from "node:assert/strict";
import { getWinLine, normalizeConfig } from "./src/utils/gameLogic.js";
import { gameReducer, createGame } from "./src/hooks/useTicTacToe.js";
const b = (s) => s.split("").map((c) => (c === "." ? null : c));
assert.deepEqual(getWinLine(b("XXX......"), 3, 3, 1).sort(), [0, 1, 2]);
assert.deepEqual(getWinLine(b("X..X..X.."), 3, 3, 6).sort((a, c) => a - c), [0, 3, 6]);
assert.deepEqual(getWinLine(b("X...X...X"), 3, 3, 4).sort((a, c) => a - c), [0, 4, 8]);
assert.deepEqual(getWinLine(b("..X.X.X.."), 3, 3, 2).sort((a, c) => a - c), [2, 4, 6]);
assert.equal(getWinLine(b("XX.......") , 3, 3, 0), null);
// no wrap across rows: 4x4, X at end of row 0 and start of row 1
assert.equal(getWinLine(b("....XXXX........"), 4, 3, 7)?.length, 4);
assert.equal(getWinLine(b("...XXX........."), 4, 3, 3), null, "must not wrap rows");
// K < N
assert.ok(getWinLine(b(".XXX............"), 4, 3, 2));
assert.deepEqual(normalizeConfig(50, 2), { size: 10, winLength: 3 });
assert.deepEqual(normalizeConfig(4, 9), { size: 4, winLength: 4 });
assert.deepEqual(normalizeConfig("", ""), { size: 3, winLength: 3 });
// reducer: play, guard, win, undo, draw
let s = createGame({ size: 3, winLength: 3 });
const play = (...idx) => idx.forEach((i) => (s = gameReducer(s, { type: "PLAY", index: i })));
play(0); const before = s; play(0); assert.equal(s, before, "filled cell ignored");
play(3, 1, 4, 2); assert.deepEqual(s.history.at(-1).winLine.sort(), [0, 1, 2]);
```

```
const won = s; play(8); assert.equal(s, won, "no moves after win");
s = gameReducer(s, { type: "UNDO" }); assert.equal(s.history.at(-1).winLine, null);
s = createGame(); play(0, 1, 2, 4, 3, 5, 7, 6, 8);
assert.equal(s.history.length - 1, 9); assert.equal(s.history.at(-1).winLine, null, "draw");
s = gameReducer(s, { type: "CONFIGURE", config: { size: 5 } });
assert.equal(s.history[0].board.length, 25); assert.equal(s.winLength, 3);
console.log("selfcheck passed");
```

## 12. Wrap-up

### Edge cases handled

- All 4 line directions, K < N, lines longer than K, no wrapping across rows.
- Win on the last cell beats draw.
- Illegal moves (filled cell, after game over) ignored in the reducer.
- Undo restores status correctly because status is derived.
- Invalid N / K input clamped.
- Keyboard-only and screen reader play.

### Trade-offs to mention

| Decision | Trade-off |
|---|---|
| Flat array vs 2D array | Simpler copies and keys; needs index ↔ (row, col) math |
| Check from last move vs full scan | O(N) per move, but relies on "only the last move can create a win", so it cannot validate an arbitrary loaded board without scanning every cell |
| Full history snapshots | $O(N^2)$ memory per move (max 100 cells × 100 moves for N = 10, tiny). For huge boards store only the move list and rebuild |
| `useReducer` vs `useState` | A bit more code, but all rules in one pure, testable function |
| Changing N/K resets the game | Simple and predictable; no "resize a game in progress" logic |
| Number input applies per keystroke | Typing "10" passes through "1" (clamped to 3) first; a debounce or apply-on-blur would fix it |

### What to add with more time

- **Redo / time travel:** keep a `step` pointer instead of slicing history.
- **Early draw detection:** stop when no line can still be completed by either player.
- **Computer player:** minimax with alpha-beta for 3×3; heuristics for bigger boards.
- **Scoreboard and persistence:** wins per player in `localStorage`.
- **Online multiplayer:** send `PLAY { index }` over WebSocket, server validates with the same reducer.
- **Tests:** React Testing Library for clicks, keyboard and status; property tests for `getWinLine`.
- **Animations** for marks and the winning line.

## 13. Counter questions the interviewer may ask

### Requirements

**Q1. Why did you ask whether K can be smaller than N?**

Because it changes the algorithm. With K = N there are exactly 2N + 2 lines and you can even use counters. With K < N, lines can start anywhere, so I need the "walk from the last move" approach. Asking first avoids rewriting the win check halfway.

**Q2. What if the interviewer says "just 3×3, keep it simple"?**

Then a hardcoded `LINES` array of 8 triples is fine and readable. I'd still keep the board as a flat array and the logic in a pure function, so upgrading to N×N later only swaps the win check.

### State design

**Q3. Why not store `currentPlayer` and `winner` in state?**

They are fully determined by the history. Storing them creates a second source of truth that must be updated in every action; forgetting it in UNDO is a classic bug. Derived values are always correct by construction.

**Q4. Why `useReducer` instead of a few `useState`s?**

A move updates board, history and win line together; the reducer does it atomically and keeps the guard ("cell filled? game over?") next to the update. It is also a pure function I can unit test in Node without rendering anything.

**Q5. Why a flat array instead of `board[row][col]`?**

One `slice()` to copy, index works as a stable React key and action payload, and `Set` lookups for the winning line are easy. The cost is the `row * N + col` conversion, which I keep in one place.

### Challenge: win check

**Q6. What is the complexity of your win check?**

O(N) per move: 4 directions, each walk at most N cells. A full-board scan would be $O(N^2 \cdot K)$. Total for a game is $O(N^3)$ in the worst case, trivial for $N \leq 10$.

**Q7. Can you make the win check O(1) per move?**

Yes, when K = N: keep counters `rows[N]`, `cols[N]`, `diag`, `antiDiag`; add +1 for X and −1 for O. A player wins when any counter reaches ±N. It does not work for K < N, which is why I chose the walk.

```
rows[r] += d; cols[c] += d;
if (r === c) diag += d;
if (r + c === n - 1) anti += d;
const won = [rows[r], cols[c], diag, anti].some((v) => Math.abs(v) === n);
```

**Q8. Why not just check `index + 1`, `index + N` in the flat array?**

`index + 1` wraps from the last column to the first column of the next row, giving fake wins. Converting to (row, col) and bounds-checking each step is the fix.

**Q9. What if the board is loaded from a saved game, not played move by move?**

Then "only the last move can win" no longer holds, so I'd run `getWinLine` for every filled cell once on load ($O(N^3)$), or replay the saved move list through the reducer, which also validates it.

## Challenge: undo and immutability

**Q10. What happens if I mutate `board[i] = "X"` directly?**

React compares by reference, so the state object looks unchanged and nothing re-renders; also every history snapshot would point to the same array, breaking undo. Always copy then set.

**Q11. How would you add redo or "jump to move 3"?**

Keep a `step` pointer next to `history`. Current snapshot is `history[step]`, undo/redo move the pointer, and a new move truncates `history.slice(0, step + 1)` before appending. Current player becomes `step % 2`.

**Q12. Isn't storing a full board per move wasteful?**

For N ≤ 10 it is at most 100 arrays of 100 cells, negligible. For a huge board I'd store only the move list `[4, 0, 8, ...]` and rebuild the board (or keep one board plus the move list and undo by clearing the last index).

## Challenge: rendering and accessibility

**Q13. How do you avoid re-rendering all $N^2$ cells on every move?**

`React.memo` on `Cell` plus stable callbacks (`useCallback` over `dispatch`). Unchanged cells get the same props and skip render. Only the played cell and, at the end, the win-line cells update.

**Q14. Why roving tabindex instead of letting every cell be tabbable?**

On a 10×10 board that is 100 Tab stops before reaching Undo. Roving tabindex makes the grid one Tab stop with arrow keys inside, which is the WAI-ARIA grid pattern users expect.

**Q15. Why `aria-disabled` and not `disabled` on finished cells?**

`disabled` removes the button from the focus order, so keyboard focus would vanish and screen reader users could not re-read the board. `aria-disabled` keeps focus while the reducer ignores the click.

## Implementation steps

**Q16. Why use the index as the `key` here? Isn't that an anti-pattern?**

Index keys are a problem when items reorder, insert or delete. Cells never move; cell 4 is always the centre. So the index is a genuinely stable id here.

**Q17. What if the user double-clicks quickly?**

Each click dispatches a separate action and the reducer always reads the latest state, so the second click hits the "cell filled" guard and returns the same state. No stale closure, because the logic is not in the click handler.

## Performance and scale

**Q18. What if N is 1000 (a million cells)?**

Rendering 1M buttons is the problem, not the win check (still O(N) per move). I'd render with a `<canvas>` or virtualize the visible window, store the board sparsely (a `Map` of filled cells) and keep the move-list history instead of snapshots.

**Q19. How would you add a computer opponent without freezing the UI?**

3×3: minimax with alpha-beta is instant. Bigger boards: heuristic scoring of open lines, and run the search in a Web Worker so the main thread stays responsive; the bot then dispatches the same `PLAY` action.

## Trade-offs and twists

**Q20. How would you make it multiplayer online?**

Server holds the authoritative state and runs the same reducer; clients send `PLAY { index }` over WebSocket and render the broadcast state. Optimistic update locally, roll back if the server rejects (not your turn, cell taken).

**Q21. What if the API to save a move is slow?**

Apply the move optimistically, show a small "saving" state, disable further input for that player until confirmed, and on failure undo the snapshot (easy, it is just popping history) and show an error.

**Q22. How would you detect a draw before the board is full?**

For every possible K-length window, check if it contains both X and O. If every window is blocked for both players, nobody can win, so it is a draw. Cost $O(N^2 \cdot K)$, so run it only after a move, or maintain counts incrementally.